



Curso de Ciência da Computação

Laboratório de POO (60h/a)

Solid Design Principles

Design Orientado a Objeto - OOD

Prof. Lenardo Chaves e Silva, D.Sc. 

lenardo@ufersa.edu.br

quinta-feira, 13 de agosto de 2026



Universidade Federal Rural do Semi-Árido (UFERSA)

Solid Design Principles

Princípios de Design Orientado a Objetos*:

- Introduzidos por Robert C. Martin (conhecido como Uncle Bob) → Engenheiro de Software e co-autor do [Manifesto Ágil](#);
- Estabelecem diretrizes (práticas) que contribuem para o desenvolvimento de software com considerações de manutenção e extensão (escalar) à medida que o projeto cresce;
- Sua adoção podem contribuir para:
 - ✓ evitar problemas de código e refatoração de código;
 - ✓ o desenvolvimento ágil e adaptativo de software.
- **SOLID:**
 - **S** - *Single-responsibility Principle* (Princípio da responsabilidade única);
 - **O** - *Open-closed Principle* (Princípio do aberto-fechado);
 - **L** - *Liskov Substitution Principle* (Princípio da substituição de Liskov);
 - **I** - *Interface Segregation Principle* (Princípio da segregação de interfaces);
 - **D** - *Dependency Inversion Principle* (Princípio da inversão de dependência).



Robert C. Martin

* FONTE: https://www.digitalocean.com/community/conceptual_articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design-pt

Solid Design Principles



SOLID:

➤ **S - Single-responsibility Principle - SRP (Princípio da responsabilidade única):**

“Uma classe deve ter apenas uma única responsabilidade, ou seja, uma única alteração na especificação do software deve ser capaz de afetar a especificação da classe.”

- Uma Classe deve ter uma, e apenas uma única razão para mudar;
- Usado para obter um fraco acoplamento entre duas funcionalidades diferentes.

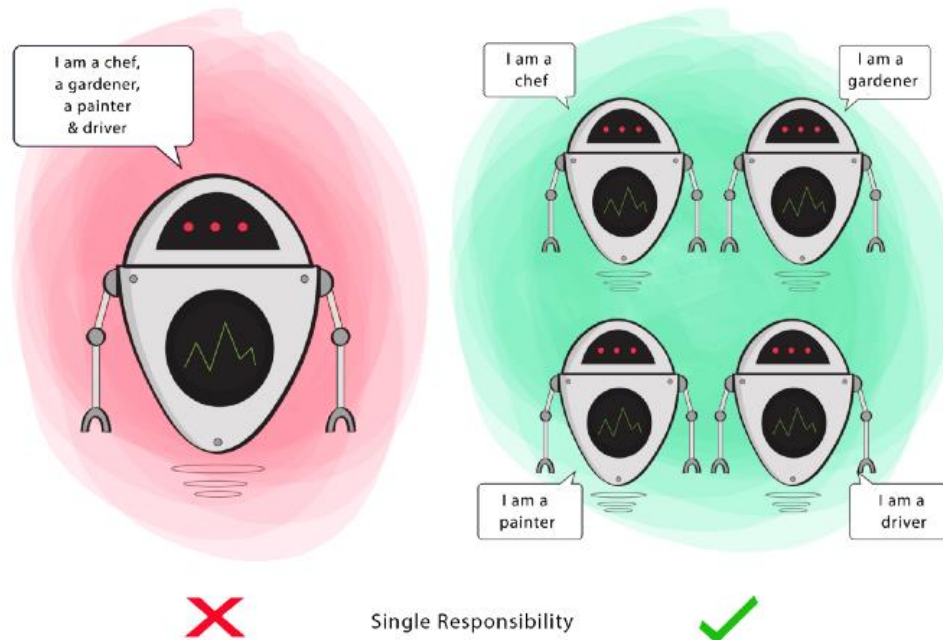
Objetivo → separar comportamentos de modo que se *bugs* surgirem como resultado de sua mudança, não afete outros comportamentos não relacionados.

Solid Design Principles



SOLID:

- **S - Single-responsibility Principle - SRP (Princípio da responsabilidade única):**



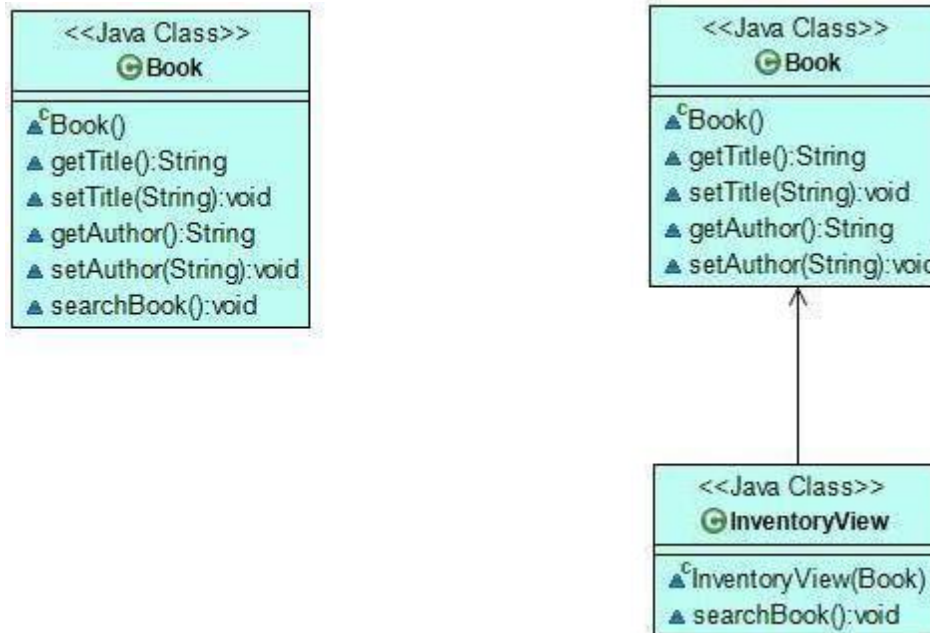
FONTE: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>

Solid Design Principles



SOLID:

- **S - Single-responsibility Principle - SRP (Princípio da responsabilidade única):**



SOLID:



➤ S - Single-responsibility Principle - SRP (Princípio da responsabilidade única):

```
class Book {
    String title, author;

    String getTitle() {
        return title;
    }

    void setTitle(String title) {
        this.title = title;
    }

    String getAuthor() {
        return author;
    }

    void setAuthor(String author) {
        this.author = author;
    }

    void searchBook() {...}
}
```



```
class Book {
    String title, author;

    String getTitle() {
        return title;
    }

    void setTitle(String title) {
        this.title = title;
    }

    String getAuthor() {
        return author;
    }

    void setAuthor(String author) {
        this.author = author;
    }
}
```

```
class InventoryView {
    Book book;

    InventoryView(Book book) {
        this.book = book;
    }

    void searchBook() {...}
}
```

Solid Design Principles



SOLID:



➤ **S - Single-responsibility Principle - SRP (Princípio da responsabilidade única):**

▪ **Onde está o problema? Vamos “codar”?**

```
public class User {
    private String name;
    private String email;

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
}
```

```
// Bad practice: Handling file storage within the User class
public void saveToFile() {
    try (FileWriter fileWriter = new FileWriter(name + ".txt")) {
        fileWriter.write("Name: " + name + "\n");
        fileWriter.write("Email: " + email + "\n");
        System.out.println("User data saved!");
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

```
public static void main(String[] args) {
    User user = new User("John Doe", "john.doe@example.com");
    user.saveToFile();
}
```



SOLID:

➤ S - Single-responsibility Principle - SRP (Princípio da responsabilidade única):

```
// User.java
public class User {
    private String name;
    private String email;

    public User(String name,String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
}

// UserFileManager.java
import java.io.FileWriter;
import java.io.IOException;

public class UserFileManager {
    public void saveToFile(User user) {
        try (FileWriter fileWriter = new FileWriter(
            user.getName() + ".txt")) {
            fileWriter.write("Name: " + user.getName() + "\n");
            fileWriter.write("Email: " + user.getEmail() + "\n");
            System.out.println("User data saved!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        User user = new User("John Doe", "john.doe@example.com");
        UserFileManager fileManager = new UserFileManager();
        fileManager.saveToFile(user);
    }
}
```

Solid Design Principles



SOLID:

➤ **O - *Open-closed Principle* (Princípio do aberto-fechado):**

“As entidades de software devem estar abertas para extensão, mas fechadas para modificação.”

- Mudar o comportamento atual de uma Classe afetará todos os sistemas que usam essa Classe;
- Usado para evitar causar *bugs* onde quer que a classe esteja sendo usada.

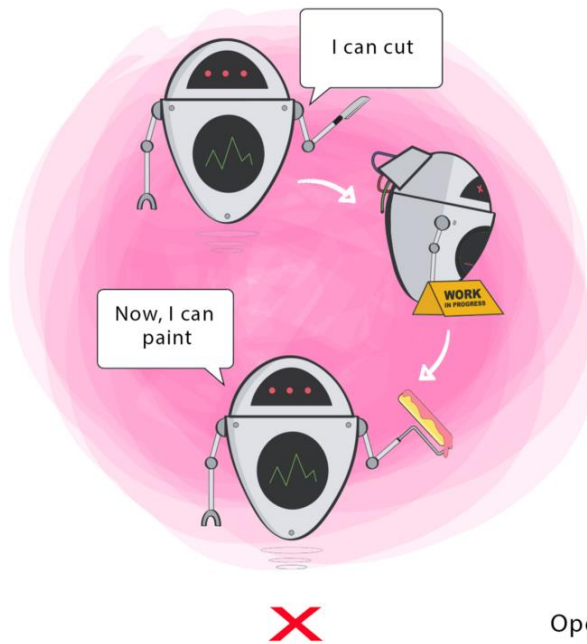
Objetivo → estender o comportamento de uma Classe sem alterar o comportamento existente dessa Classe.

Solid Design Principles

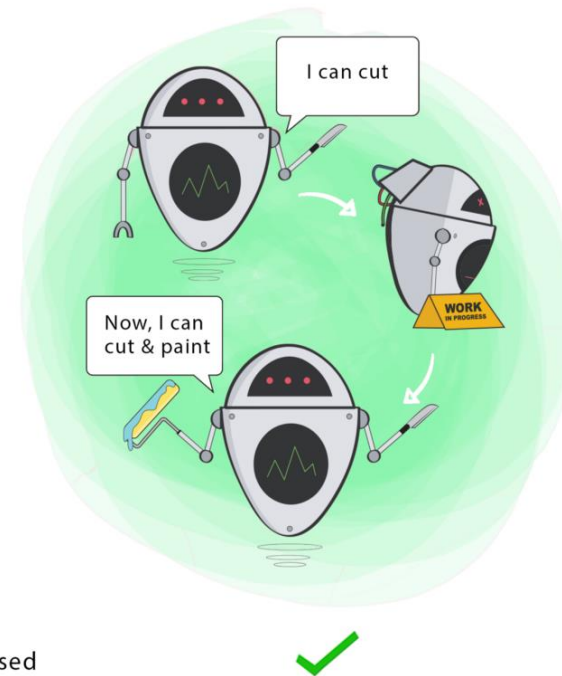


SOLID:

- **O - Open-closed Principle (Princípio do aberto-fechado):**



Open-Closed

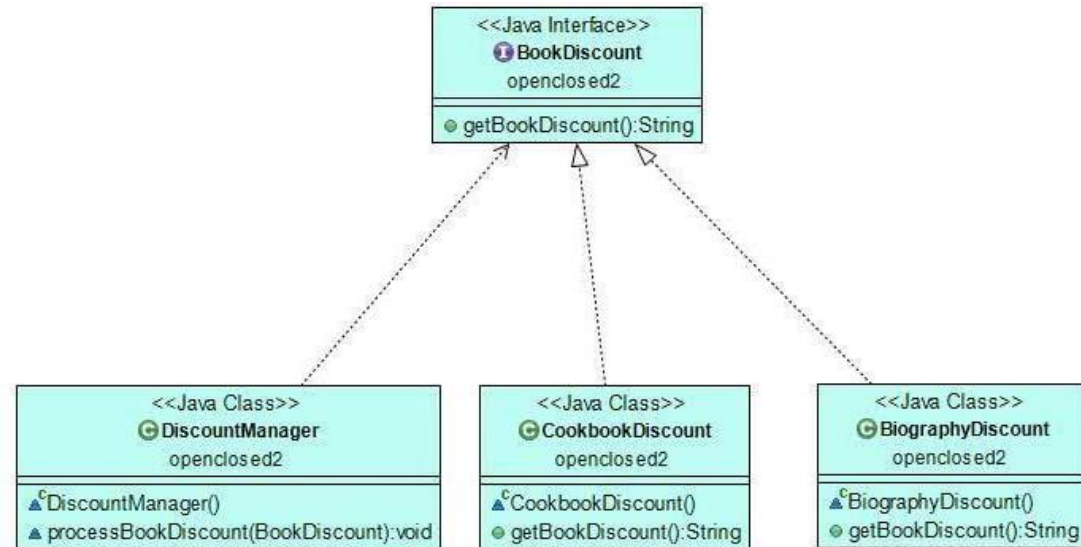
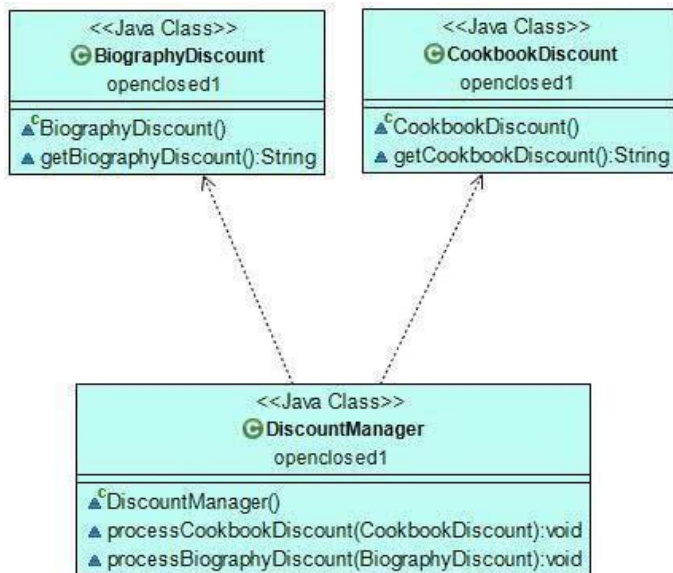


FONTE: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>



SOLID:

➤ O - Open-closed Principle (Princípio do aberto-fechado):





SOLID:

➤ O - Open-closed Principle (Princípio do aberto-fechado):

```
class CookbookDiscount {
    String getCookbookDiscount() {
        String discount = "30% between Dec 1 and 24.";
        return discount;
    }
}

class DiscountManager {
    void processCookbookDiscount(CookbookDiscount d) {
        //...
    }
}
```

//Now, biography books with a 50% discount on the subject's birthday.

```
class BiographyDiscount {
    String getBiographyDiscount() {
        String discount = "50% on the subject's birthday.";
        return discount;
    }
}
```

//We need to add the new functionality to the DiscountManager class, too:

```
class DiscountManager {
    void processCookbookDiscount(CookbookDiscount d){
        //...
    }

    void processBiographyDiscount(BiographyDiscount d){
        //...
    }
}
```



SOLID:

➤ O - Open-closed Principle (Princípio do aberto-fechado):

//Adding an extra layer of abstraction
that represents all types of discounts.

```
public interface BookDiscount {  
  
    String getBookDiscount();  
  
}
```

```
class CookbookDiscount implements BookDiscount {  
  
    @Override  
    public String getBookDiscount() {  
        String discount = "30% between Dec 1 and 24.";  
        return discount;  
    }  
}
```

```
class BiographyDiscount implements BookDiscount {  
  
    @Override  
    public String getBookDiscount() {  
        String discount = "50% on the subject's birthday.";  
        return discount;  
    }  
}
```

//Now, DiscountManager can refer to
the BookDiscount interface instead of the
concrete classes.

```
class DiscountManager {  
  
    void processBookDiscount(BookDiscount d) {  
        \\...  
    }  
}
```

Solid Design Principles



SOLID:

➤ O - *Open-closed Principle* (Princípio do aberto-fechado):

▪ *Onde está o problema? Vamos “codar”?*

```
// DiscountCalculator.java
public class DiscountCalculator {
    public double calculateDiscount(String customerType, double amount) {
        if (customerType.equals("Regular")) {
            return amount * 0.1;
        } else if (customerType.equals("Premium")) {
            return amount * 0.2;
        }
        return 0;
    }

    public static void main(String[] args) {
        DiscountCalculator calculator = new DiscountCalculator();
        double regularDiscount = calculator.calculateDiscount("Regular", 100);
        double premiumDiscount = calculator.calculateDiscount("Premium", 100);

        System.out.println("Regular Discount: " + regularDiscount);
        System.out.println("Premium Discount: " + premiumDiscount);
    }
}
```



SOLID:

➤ O - Open-closed Principle (Princípio do aberto-fechado):

```
// Discount.java
public abstract class Discount {
    public abstract double calculate(double amount);
}
```

```
// RegularDiscount.java
public class RegularDiscount extends Discount {
    public double calculate(double amount) {
        return amount * 0.1;
    }
}
```

```
// PremiumDiscount.java
public class PremiumDiscount extends Discount {
    public double calculate(double amount) {
        return amount * 0.2;
    }
}
```

```
// DiscountCalculator.java
public class DiscountCalculator {
    public double calculateDiscount(Discount discount, double amount) {
        return discount.calculate(amount);
    }
}
```

```
// Main.java
public class Main {
    public static void main(String[] args) {
        Discount regularDiscount = new RegularDiscount();
        Discount premiumDiscount = new PremiumDiscount();

        DiscountCalculator calculator = new
            DiscountCalculator();

        double regularDiscountAmount =
            calculator.calculateDiscount(regularDiscount, 100);

        double premiumDiscountAmount =
            calculator.calculateDiscount(premiumDiscount, 100);

        System.out.println("Regular Discount: " +
            regularDiscountAmount);
        System.out.println("Premium Discount: " +
            premiumDiscountAmount);
    }
}
```

Solid Design Principles



SOLID:

➤ **L - *Liskov Substitution Principle** (Princípio da substituição de Liskov):**

“Os objetos em um programa devem ser substituídos por instâncias de seus subtipos sem alterar a corretude desse programa.”

- Manter a padronização comportamental das subclasses em relação à sua superclasse;
- Usado para evitar causar *bugs* em virtude do uso inadequado do conceito de *Herança*.

Objetivo → impor consistência para que a Classe mãe ou sua Classe filha possam ser usadas de maneira semelhantes sem erros.

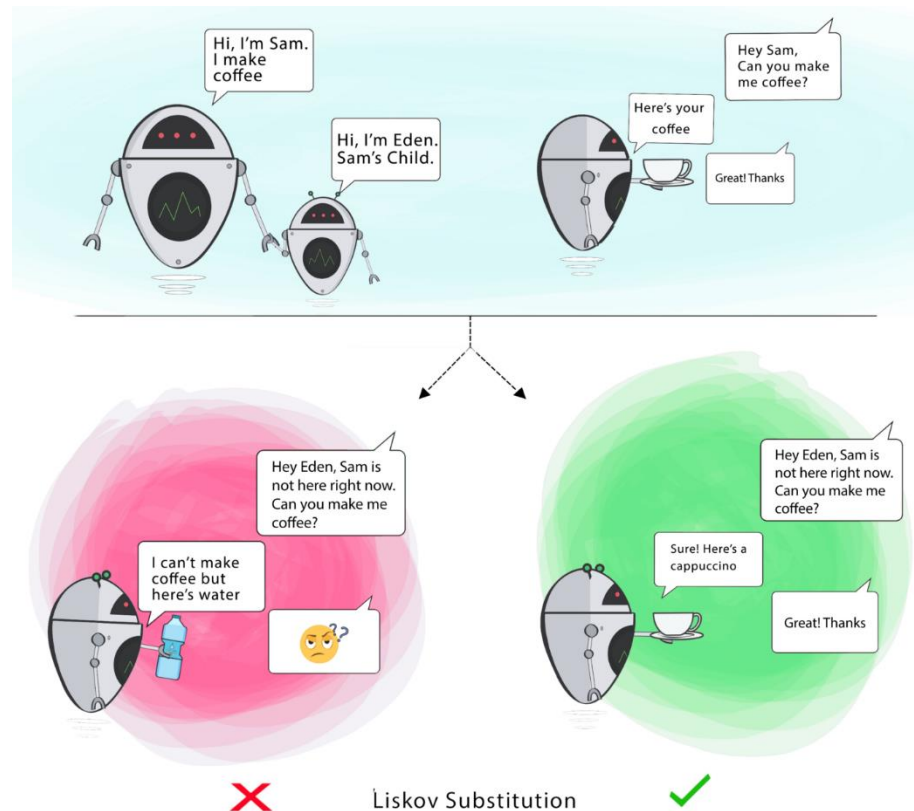
**It was Barbara Liskov who introduced the principle in 1987 in her conference keynote talk “Data Abstraction”.*

Solid Design Principles



SOLID:

➤ L - Liskov Substitution Principle (Princípio da substituição de Liskov):



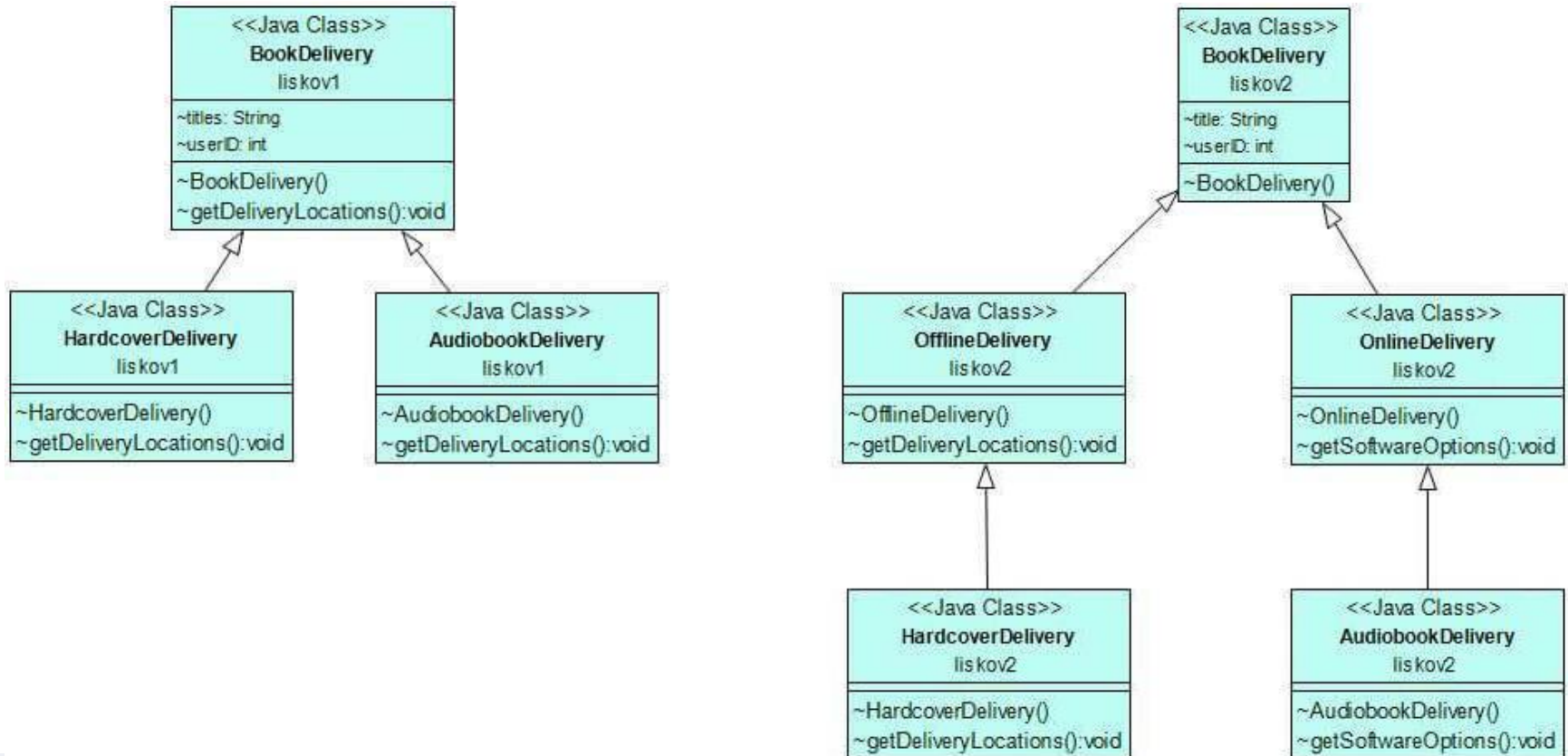
FONTE: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>

Solid Design Principles



SOLID:

- **L - Liskov Substitution Principle** (Princípio da substituição de Liskov):



Solid Design Principles



SOLID:

➤ L - Liskov Substitution Principle (Princípio da substituição de Liskov):

```
//It informs customers about the number of locations where
they can collect their order:
```

```
class BookDelivery {
    String titles;
    int userID;

    void getDeliveryLocations() {...}
}
```

```
//The store also sells hardcovers:
```

```
class HardcoverDelivery extends BookDelivery {

    @Override
    void getDeliveryLocations() {...}

}
```

```
//The store asks us to create delivery
functionalities for audiobooks, too:
```

```
class AudiobookDelivery extends BookDelivery {

    @Override
    void getDeliveryLocations() {
        /* can't be implemented */
    }

}
```

Audiobooks can't be delivered to physical locations.

Solid Design Principles



SOLID:

➤ L - Liskov Substitution Principle (Princípio da substituição de Liskov):

//To solve the problem, we need to fix the inheritance hierarchy, introducing an extra layer that better differentiates book delivery types:

```
class BookDelivery {
    String title;
    int userID;
}
```

// We also move the getDeliveryLocations() method for the OfflineDelivery class:

```
class OfflineDelivery extends BookDelivery {
    void getDeliveryLocations() {...}
}
```

// HardcoverDelivery will be the child class of OfflineDelivery :

```
class HardcoverDelivery extends OfflineDelivery {
    @Override
    void getDeliveryLocations() {...}
}
```

//We create a new getSoftwareOptions() method for the OnlineDelivery class:

```
class OnlineDelivery extends BookDelivery {
    void getSoftwareOptions() {...}
}
```

//AudiobookDelivery will be the child class of OnlineDelivery:

```
class AudiobookDelivery extends OnlineDelivery {
    @Override
    void getSoftwareOptions() {...}
}
```

Solid Design Principles



SOLID:



➤ L - Liskov Substitution Principle (Princípio da substituição de Liskov):

```
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}
```

```
public class Square extends Rectangle {
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }

    public static void main(String[] args) {
        Rectangle rectangle = new Rectangle();
        rectangle.setWidth(5);
        rectangle.setHeight(10);
        System.out.println("Rectangle Area: " +
            rectangle.getArea());

        Square square = new Square();
        square.setWidth(5);
        System.out.println("Square Area: " +
            square.getArea());
    }
}
```



SOLID:

➤ L - *Liskov Substitution Principle* (Princípio da substituição de Liskov):

```
// Shape.java
public interface Shape {
    int getArea();
}

// Rectangle.java
public class Rectangle implements Shape {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}
```

```
// Square.java
public class Square implements Shape {
    private int side;

    public Square(int side) {
        this.side = side;
    }

    public int getArea() {
        return side * side;
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        Shape rectangle = new Rectangle(5, 10);
        Shape square = new Square(5);

        System.out.println("Rectangle Area: " +
                           rectangle.getArea());
        System.out.println("Square Area: " +
                           square.getArea());
    }
}
```

Solid Design Principles



SOLID:

➤ **I - Interface Segregation Principle (Princípio da segregação de interfaces):**

“Clientes (classes) não devem ser forçados a implementar uma interface (ou a depender de métodos) que ele não usa.”

- Executar apenas as ações necessárias para cumprir sua função;
- Usado para evitar desperdícios em uma Classe por conta da sua incapacidade de executar ações (de uma interface) que não têm utilidade para ela.

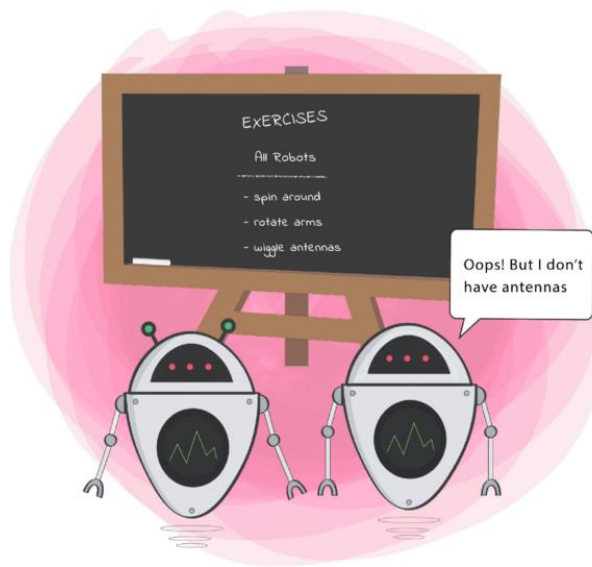
Objetivo → dividir um conjunto de ações em conjuntos menores para que uma Classe execute apenas o conjunto de ações que ela necessita.

Solid Design Principles

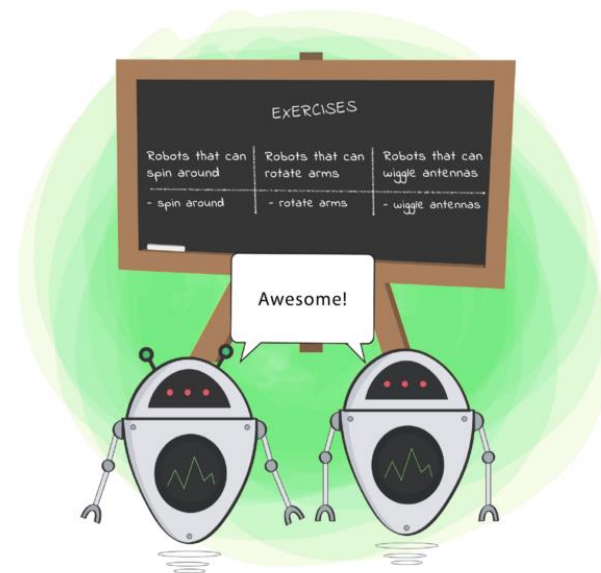


SOLID:

➤ I - *Interface Segregation Principle* (Princípio da segregação de interfaces):



Interface Segregation



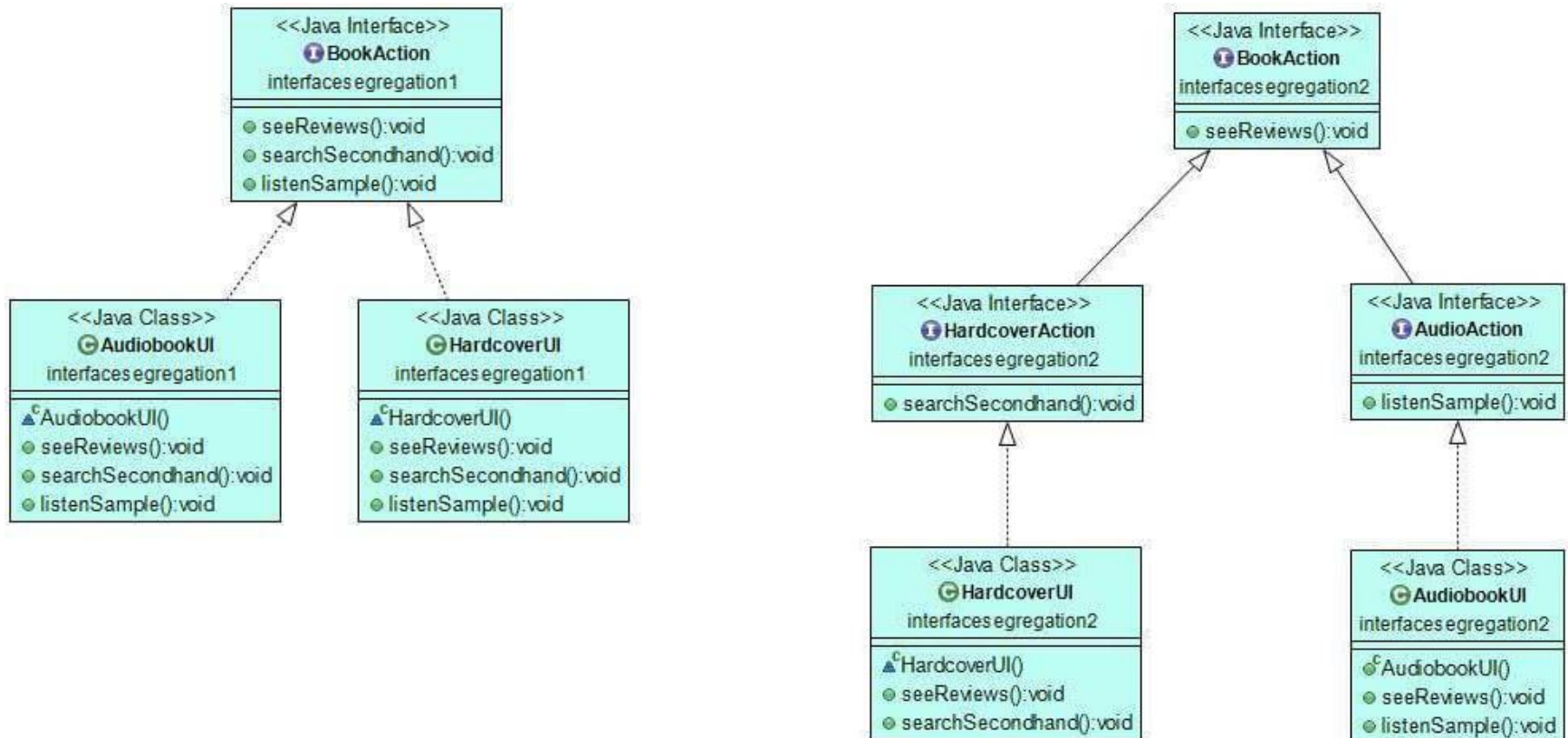
FONTE: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>

Solid Design Principles



SOLID:

- **I - Interface Segregation Principle (Princípio da segregação de interfaces):**



Solid Design Principles



SOLID:

➤ I - Interface Segregation Principle (Princípio da segregação de interfaces):

//Adding some user actions to our online bookstore so that customers can interact with the content before making a purchase:

```
public interface BookAction {

    void seeReviews();
    void searchSecondhand();
    void listenSample();

}
```

//HardcoverUI implements the BookAction interface with its own functionalities:

```
class HardcoverUI implements BookAction {

    @Override
    public void seeReviews() {...}

    @Override
    public void searchSecondhand() {...}

    @Override
    public void listenSample() {...}

}
```

//AudiobookUI implements the BookAction interface with its own functionalities:

```
class AudiobookUI implements BookAction {

    @Override
    public void seeReviews() {...}

    @Override
    public void searchSecondhand() {...}

    @Override
    public void listenSample() {...}

}
```

Both classes depend on methods they don't use!

Solid Design Principles



SOLID:

➤ I - Interface Segregation Principle (Princípio da segregação de interfaces):

//Let's extend the BookAction interface with two more specific sub-interfaces: HardcoverAction and AudioAction:

```
public interface BookAction {  
    void seeReviews();  
}
```

```
public interface HardcoverAction extends BookAction {  
    void searchSecondhand();  
}
```

```
public interface AudioAction extends BookAction {  
    void listenSample();  
}
```

```
class HardcoverUI implements HardcoverAction {  
    @Override  
    public void seeReviews() {...}  
    @Override  
    public void searchSecondhand() {...}  
}
```

```
class AudiobookUI implements AudioAction {  
    @Override  
    public void seeReviews() {...}  
    @Override  
    public void listenSample() {...}  
}
```

Now, both classes don't have to implement irrelevant methods!

Solid Design Principles



SOLID:

➤ I - *Interface Segregation Principle* (Princípio da segregação de interfaces):

```
public interface Printer {  
    void printDocument();  
    void scanDocument();  
    void faxDocument();  
}  
  
public class SimplePrinter implements Printer {  
    public void printDocument() {  
        System.out.println("Printing document...");  
    }  
  
    public void scanDocument() {  
        // Not implemented  
    }  
  
    public void faxDocument() {  
        // Not implemented  
    }  
  
    public static void main(String[] args) {  
        SimplePrinter printer = new SimplePrinter();  
        printer.printDocument();  
    }  
}
```

SOLID:



➤ I - Interface Segregation Principle (Princípio da segregação de interfaces):

```
// Printer.java
public interface Printer {
    void printDocument();
}

// Scanner.java
public interface Scanner {
    void scanDocument();
}

// Fax.java
public interface Fax {
    void faxDocument();
}

// SimplePrinter.java
public class SimplePrinter implements Printer {
    public void printDocument() {
        System.out.println("Printing document...");
    }
}
```

```
// MultiFunctionPrinter.java
public class MultiFunctionPrinter implements Printer,
Scanner, Fax {

    public void printDocument() {
        System.out.println("Printing document...");
    }

    public void scanDocument() {
        System.out.println("Scanning document...");
    }

    public void faxDocument() {
        System.out.println("Faxing document...");
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        Printer simplePrinter = new SimplePrinter();
        simplePrinter.printDocument();

        MultiFunctionPrinter mfp = new MultiFunctionPrinter();
        mfp.printDocument();
        mfp.scanDocument();
        mfp.faxDocument();
    }
}
```

Solid Design Principles



SOLID:

➤ **D - *Dependency Inversion Principle* (Princípio da inversão de dependência):**

“Deve-se depender de abstrações, em vez de implementações concretas.”

- Separar as Classes (ou Módulos) umas das outras por meio de uma camada de abstração (Interface) que permita interligá-las ;
- Usado para garantir a conexão adequada entre duas Classes que não se conhecem (não sabem como cada uma funciona).

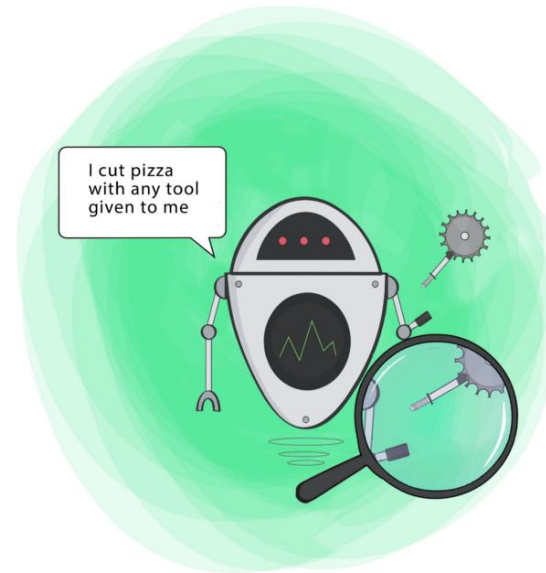
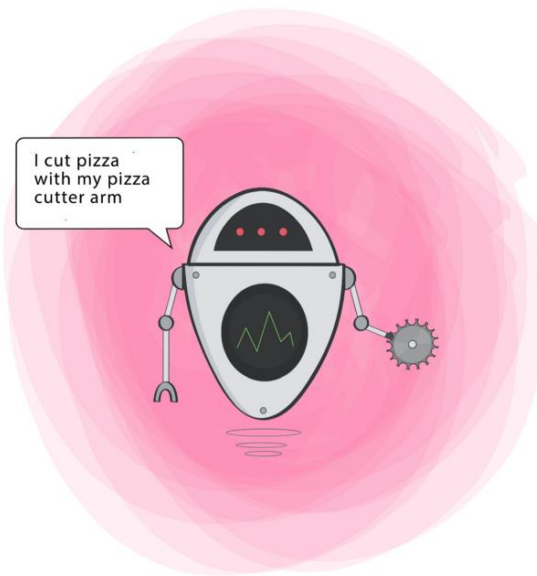
Objetivo → reduzir a dependência de uma Classe de alto nível (aquela que executa uma ação com uma ferramenta) na Classe de baixo nível (aquela que é necessária para executar uma ação) por introduzir uma Interface.

Solid Design Principles



SOLID:

- **D - Dependency Inversion Principle** (Princípio da inversão de dependência):



Dependency Inversion

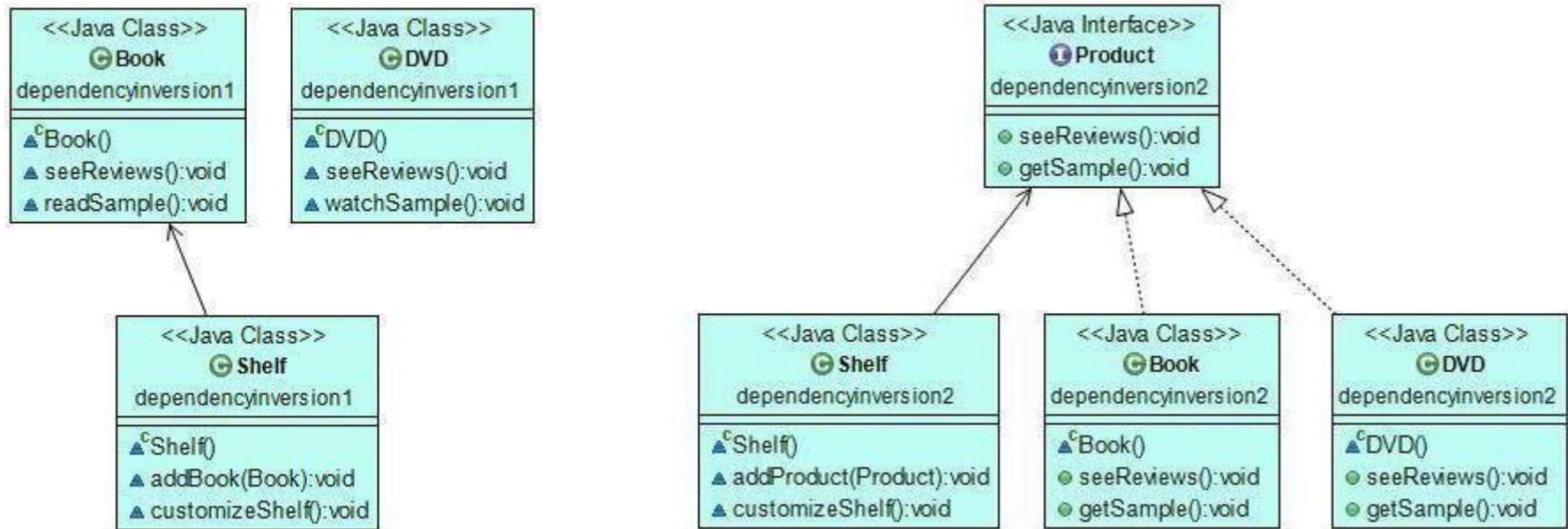
FONTE: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>

Solid Design Principles



SOLID:

- **D - Dependency Inversion Principle** (Princípio da inversão de dependência):





SOLID:

➤ D - *Dependency Inversion Principle* (Princípio da inversão de dependência):

//Now, the bookstore asked us to build a new feature that enables customers to put their favorite books on a shelf:

```
class Book {  
  
    void seeReviews() {...}  
    void readSample() {...}  
}
```

The high-level Shelf class depends on the low-level Book!

```
class Shelf {  
    Book book;  
  
    void addBook(Book book) {...}  
    void customizeShelf() {...}  
}
```

//The Book class (lower-level) will allow users to see reviews and read a sample of each book they store on their shelves:

//The Shelf (high-level) class will let them add a book to their shelf and customize the shelf.:

Now, we should modify the Shelf class so that it can accept DVDs, too!

```
class DVD {  
  
    void seeReviews() {...}  
    void watchSample() {...}  
}
```

//The **problem** becomes clear when the store asks us to enable customers to add DVDs to their shelves, too:

Solid Design Principles



SOLID:

➤ D - *Dependency Inversion Principle* (Princípio da inversão de dependência):

//We need to create an abstraction layer for the lower-level classes (Book and DVD):

```
public interface Product {  
  
    void seeReviews();  
    void getSample();  
  
}
```



```
class Book implements Product {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void getSample() {...}  
  
}
```



```
class DVD implements Product {  
  
    @Override  
    public void seeReviews() {...}  
  
    @Override  
    public void getSample() {...}  
  
}
```

//We'll do so by introducing the Product interface that both classes will implement.

Solid Design Principles



SOLID:

➤ D - *Dependency Inversion Principle* (Princípio da inversão de dependência):

//Now, Shelf can reference the Product interface instead of its implementations (Book and DVD):

```
class Shelf {  
  
    Product product;  
  
    void addProduct(Product product) {...}  
  
    void customizeShelf() {...}  
  
}
```

//The refactored code also allows us to later introduce new product types (for instance, Magazine) that customers can put on their shelves, too.

Solid Design Principles



SOLID:

➤ D - *Dependency Inversion Principle* (Princípio da inversão de dependência):

```
public class LightBulb {  
    public void turnOn() {  
        System.out.println("LightBulb on");  
    }  
  
    public void turnOff() {  
        System.out.println("LightBulb off");  
    }  
}
```

```
public class Switch {  
    private LightBulb lightBulb;  
  
    public Switch(LightBulb lightBulb) {  
        this.lightBulb = lightBulb;  
    }  
  
    public void operate() {  
        lightBulb.turnOn();  
    }  
  
    public static void main(String[] args) {  
        LightBulb lightBulb = new LightBulb();  
        Switch lightSwitch = new Switch(lightBulb);  
        lightSwitch.operate();  
    }  
}
```

SOLID:



➤ D - *Dependency Inversion Principle* (Princípio da inversão de dependência):

```
// Switchable.java
public interface Switchable {
    void turnOn();
    void turnOff();
}

// LightBulb.java\
public class LightBulb implements Switchable {
    public void turnOn() {
        System.out.println("LightBulb on");
    }

    public void turnOff() {
        System.out.println("LightBulb off");
    }
}
```

```
// Switch.java
public class Switch {
    private Switchable device;

    public Switch(Switchable device) {
        this.device = device;
    }

    public void operate() {
        device.turnOn();
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        Switchable lightBulb = new LightBulb();
        Switch lightSwitch = new Switch(lightBulb);
        lightSwitch.operate();
    }
}
```

Solid Design Principles



Por quê implementar os Princípios DOO:

- Aumenta a complexidade global do código base, mas leva a um projeto mais flexível;
- Podem ser adotados em aplicações monolíticas, bem como para os microserviços (cada microserviço como um módulo de código autônomo);
- Na quebra de um princípio de projeto SOLID, Java e outras linguagens compiladas podem lançar uma exceção, mas isso nem sempre acontece.